Universidad tecnológica de San Luis Rio Colorado

Desarrollo web integral

José Julián Reyes Patiño

Ingeniería en desarrollo y gestión de software

Hernández Acosta Héctor Manuel 9-2

Unidad 3- Practica 4: Backend con Django y JWT (Local)

08 de noviembre de 2026

Repositorio
https://github.com/Hector9229/tienda-online-django.git

1. Datos del proyecto

Se tomó como base el proyecto de la Unidad 1 (Django + DRF + CRUD de productos) y se adaptó a los requisitos de la práctica. El proyecto se llama tienda_online y el tema es una tienda en línea; el título de Swagger es Tienda Online API.

Requisito	Implementación
Nombre y tema propios	tienda_online — tienda en línea
Título de Swagger	Tienda Online API
Campo extra del usuario	phone (teléfono), visible en admin, serializer y Swagger
Login	Email + contraseña (se eliminó username)
Token de acceso	ACCESS_TOKEN_LIFETIME = 60 minutos (configurable por variable de entorno)
Documentación	Swagger UI en /swagger/ y ReDoc en /redoc/
Base de datos	SQLite (db.sqlite3 excluido en .gitignore)

2. Campo extra del usuario: teléfono (phone)

Sí se agregó un campo extra al modelo de usuario: phone (Teléfono). Es un CharField de hasta 20 caracteres y se muestra en el admin de Django, en el serializer y en la documentación Swagger.

2.1. En el modelo

```
class User(AbstractUser):
    username = None

    email = models.EmailField('Correo electrónico', unique=True)
    phone = models.CharField(
        'Teléfono',
        max_length=20,
        blank=True,
        help_text='Número telefónico de contacto (10 dígitos).',
    )

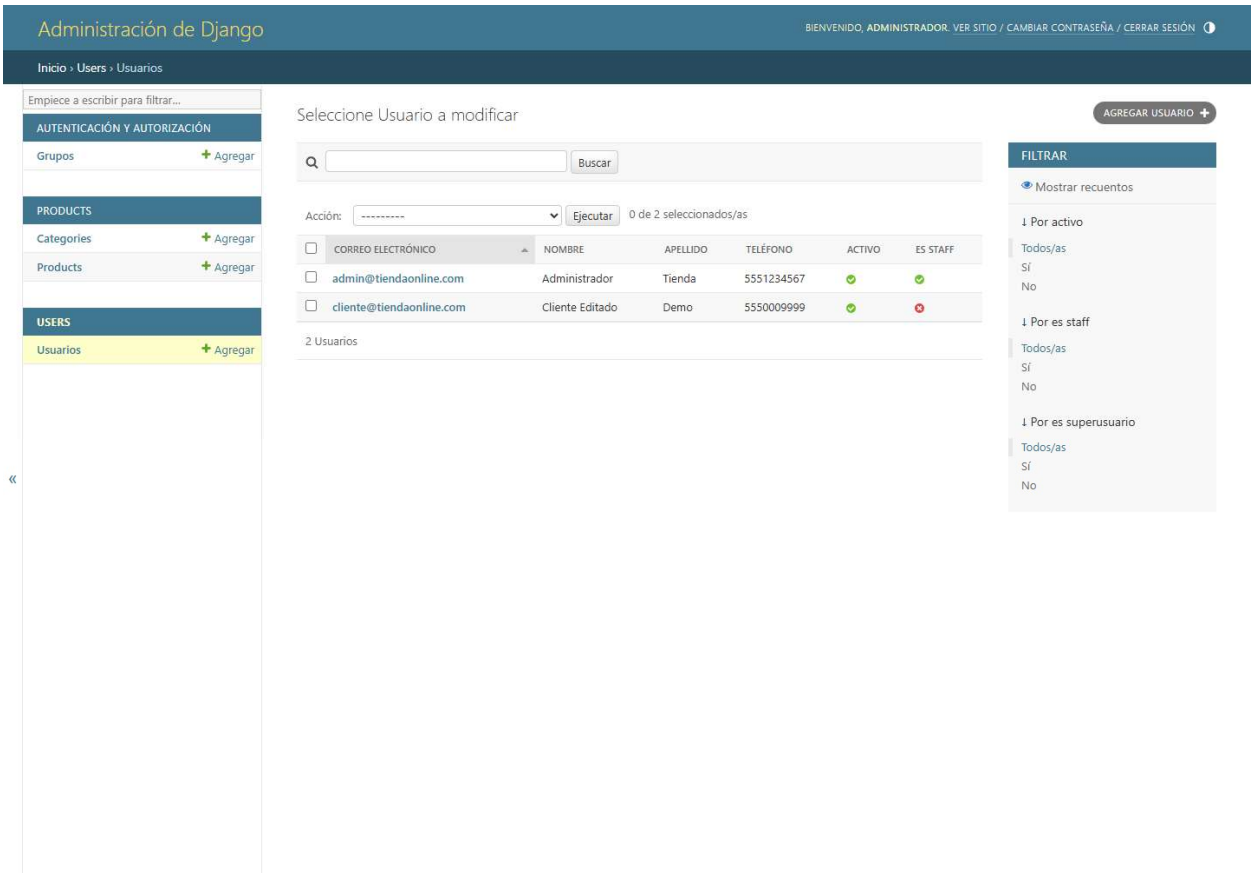
    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = []
```

2.2. En el admin de Django

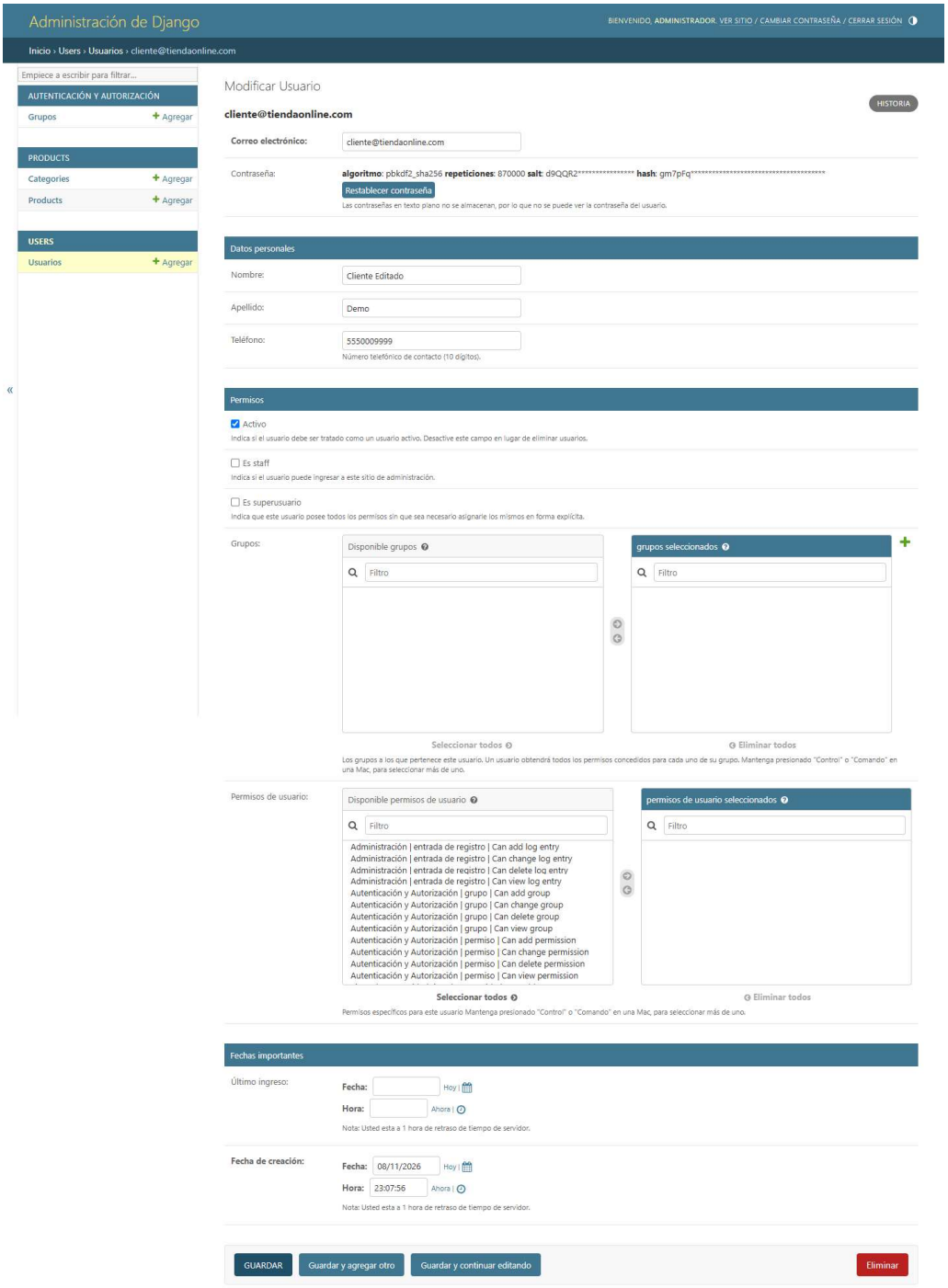
El campo aparece en el listado de usuarios (columna Teléfono), en el formulario de edición (sección Datos personales), en el formulario de alta y en la búsqueda del admin.

```
list_display = ('email', 'first_name', 'last_name', 'phone',
                'is_active', 'is_staff')
search_fields = ('email', 'first_name', 'last_name', 'phone')
fieldsets = (
```

```
    ...
    ('Datos personales', {'fields': ('first_name', 'last_name', 'phone')}),
    ...
)
```



Captura A. Admin de Django: listado de usuarios con la columna Teléfono (campo extra solicitado).



Captura B. Admin de Django: detalle del usuario con el campo Teléfono visible en el formulario.

2.3. En el serializer

El campo phone está incluido en los campos que expone la API, por lo que se puede enviar y recibir al crear o consultar usuarios.

```
class Meta:
    model = User
    fields = (
        'id', 'email', 'first_name', 'last_name', 'phone',
        'password', 'is_active', 'is_staff', 'date_joined',
    )
    read_only_fields = ('id', 'date_joined')
```

2.4. En Swagger

Al consultar el esquema OpenAPI (/swagger/?format=openapi) la definición User incluye la propiedad phone. También aparece en el cuerpo de ejemplo de POST /users/ y en la respuesta de /auth/me/.

```
definitions.User.properties =
['id', 'email', 'first_name', 'last_name', 'phone', 'password',
 'is_active', 'is_staff', 'date_joined']
```

El campo password aparece en la definición únicamente como dato de entrada (write-only); nunca se devuelve en las respuestas.

2.5. Cómo comprobarlo

1. Abrir el admin en http://127.0.0.1:8000/admin/ → Usuarios: se ve la columna Teléfono y, al editar un usuario, el campo dentro de Datos personales (capturas A y B).
2. Abrir http://127.0.0.1:8000/swagger/ → users → POST /users/: el cuerpo de ejemplo incluye phone. También se puede abrir http://127.0.0.1:8000/swagger/?format=openapi y buscar "phone" en definitions.User.
3. Hacer POST /api/v1/users/ con un phone y luego GET /api/v1/users/{id}/: el valor se guarda y se devuelve.
4. Hacer GET /api/v1/auth/me/ con el token: la respuesta incluye phone (ver captura 3).

3. Tiempo de vida del token de acceso

Sí se configuró ACCESS_TOKEN_LIFETIME en la configuración de SimpleJWT dentro de tienda_online/settings.py. El valor por defecto es de 60 minutos y puede cambiarse con la variable de entorno ACCESS_TOKEN_LIFETIME_MINUTES sin modificar el código.

```
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(
        minutes=int(os.environ.get('ACCESS_TOKEN_LIFETIME_MINUTES', 60))
    ),
    'REFRESH_TOKEN_LIFETIME': timedelta(
        days=int(os.environ.get('REFRESH_TOKEN_LIFETIME_DAYS', 1))
    ),
    'ROTATE_REFRESH_TOKENS': False,
    'BLACKLIST_AFTER_ROTATION': False,
    'AUTH_HEADER_TYPES': ('Bearer',),
}
```

3.1. Comprobación realizada

Se obtuvo un token con POST /api/v1/auth/token/ y se decodificó su payload. La diferencia entre exp (expiración) e iat (emisión) es exactamente 3600 segundos, es decir, 60 minutos:

Dato del token	Valor
Emisión (iat)	2026-11-09 05:31:09 UTC
Expiración (exp)	2026-11-09 06:31:09 UTC
Diferencia (exp − iat)	3600 segundos = 60 minutos

3.2. Cómo comprobarlo

Opción 1: con el token en la mano.

5. Ejecutar POST /api/v1/auth/token/ con email y contraseña.
6. Copiar el valor de access.
7. Abrirlo en https://jwt.io o decodificarlo localmente.
8. Comparar iat y exp: la diferencia debe ser de 3600 segundos (60 minutos).

Opción 2: con un script de Python (mismo cálculo que se usó).

```
import base64, json

payload = access.split('.')[1]
payload += '=' * (-len(payload) % 4)
data = json.loads(base64.urlsafe_b64decode(payload))
print(data['exp'] - data['iat'])   # 3600
```

Opción 3: cambiar la duración sin tocar el código.

```
# Windows PowerShell
$env:ACCESS_TOKEN_LIFETIME_MINUTES=5
python manage.py runserver

# Al pedir un token nuevo, exp - iat sera 300 segundos (5 minutos).
```

4. Correcciones de seguridad

El manual original incluía dos fallas de seguridad. Ambas fueron corregidas en el serializer de usuarios.

4.1. La contraseña se devolvía en las respuestas

El serializer incluía password como campo normal, por lo que el hash aparecía en el CRUD y en /auth/me/. Se declaró como write_only=True: se puede enviar al crear o actualizar, pero nunca se incluye en las respuestas.

```
password = serializers.CharField(
    write_only=True,
    required=False,
    allow_blank=False,
    min_length=8,
    style={'input_type': 'password'},
)
```

4.2. Editar sin contraseña cambiaba la contraseña

El manual asignaba request.user.password cuando la petición no traía contraseña. Eso cambiaba la contraseña del usuario editado por el hash del administrador que hacía la petición. Ahora la lógica está en el serializer y solo se llama a set_password() si la contraseña viene en la petición.

```
def update(self, instance, validated_data):
    password = validated_data.pop('password', None)

    for field, value in validated_data.items():
        setattr(instance, field, value)

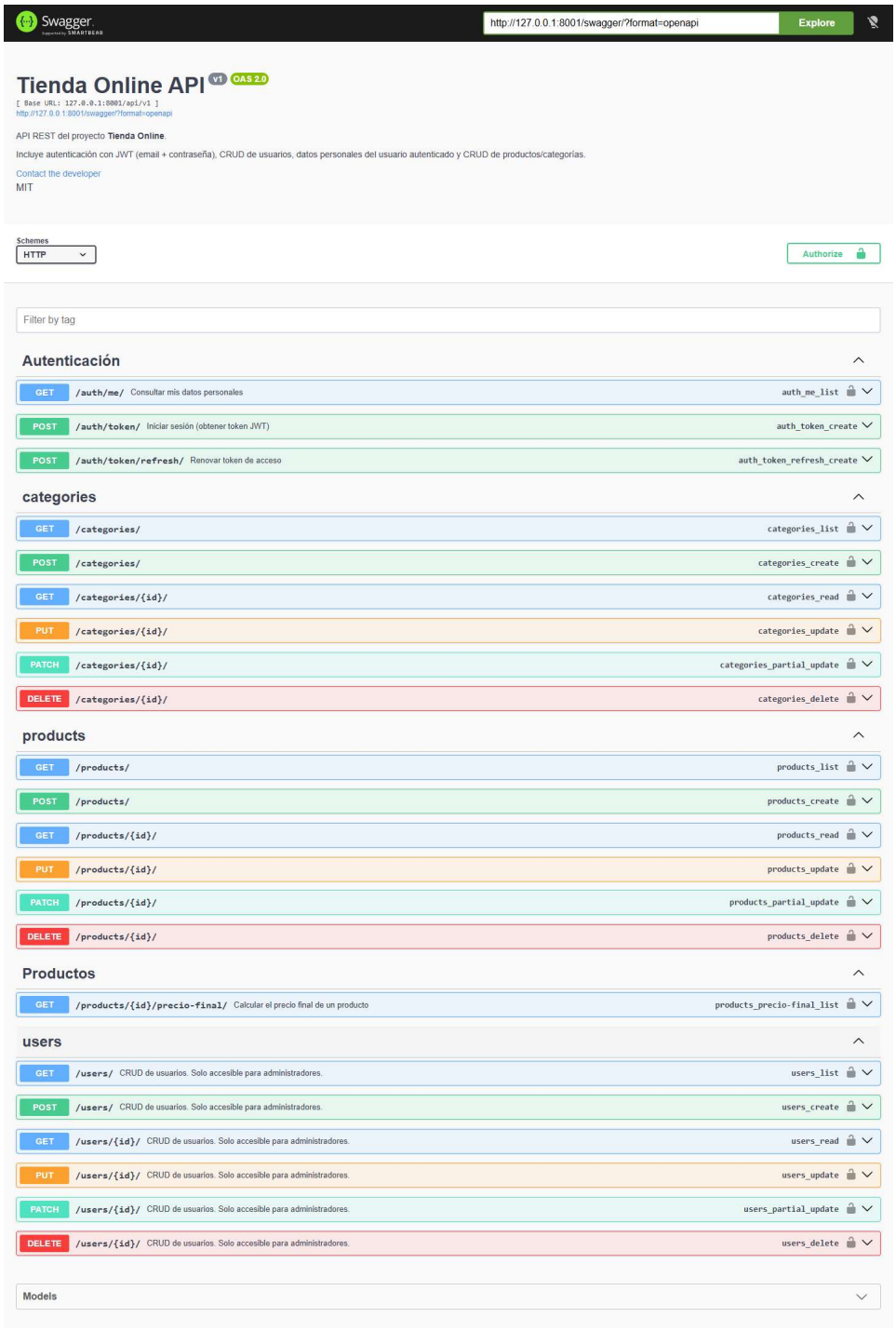
    if password:
        instance.set_password(password)

    instance.save()
    return instance
```

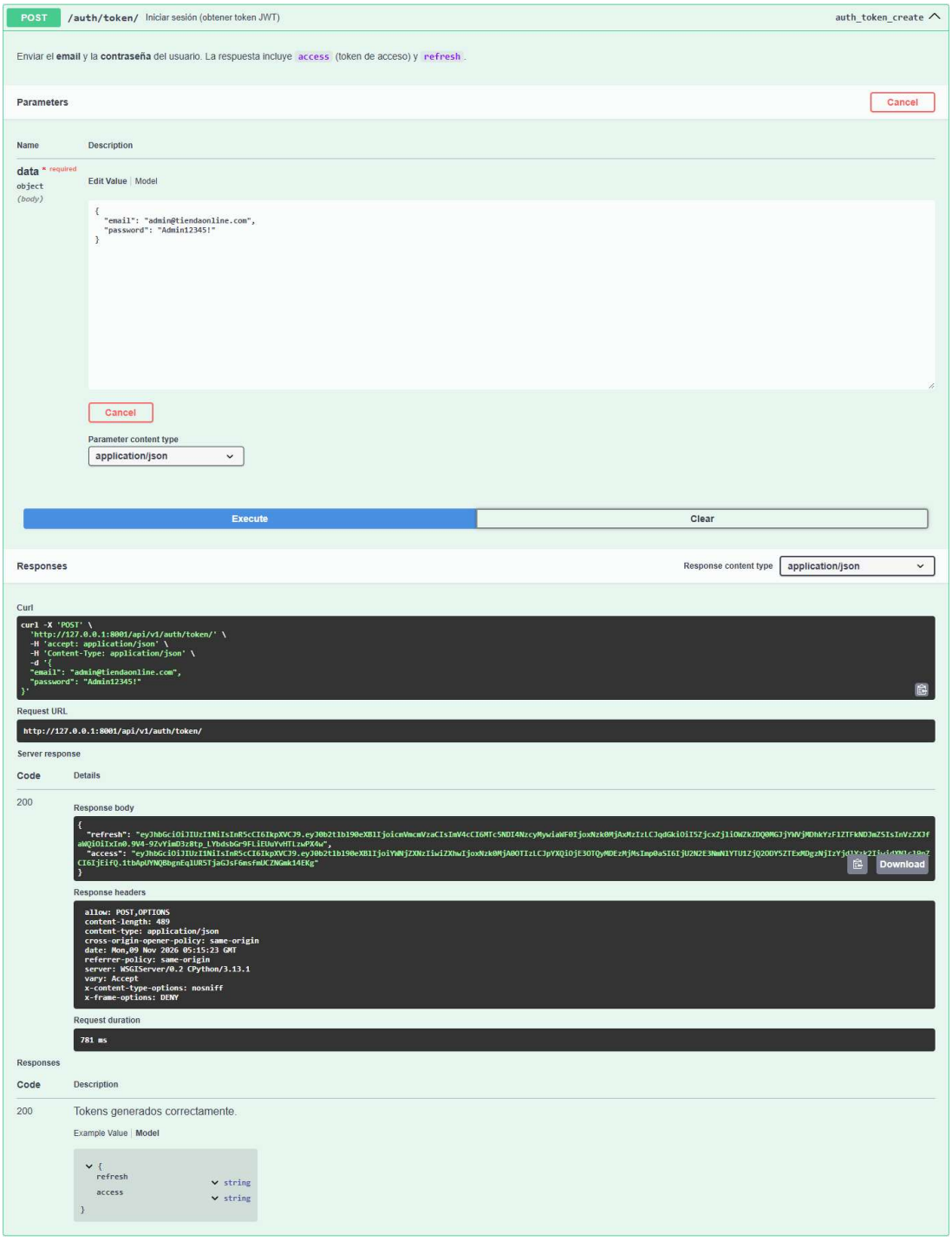
La captura 4 muestra un PATCH /api/v1/users/2/ sin el campo password y la captura 5 muestra la comprobación del hash: antes y después del PATCH es idéntico, check_password() devuelve True y la prueba automatizada termina en OK.

5. Capturas de pantalla requeridas

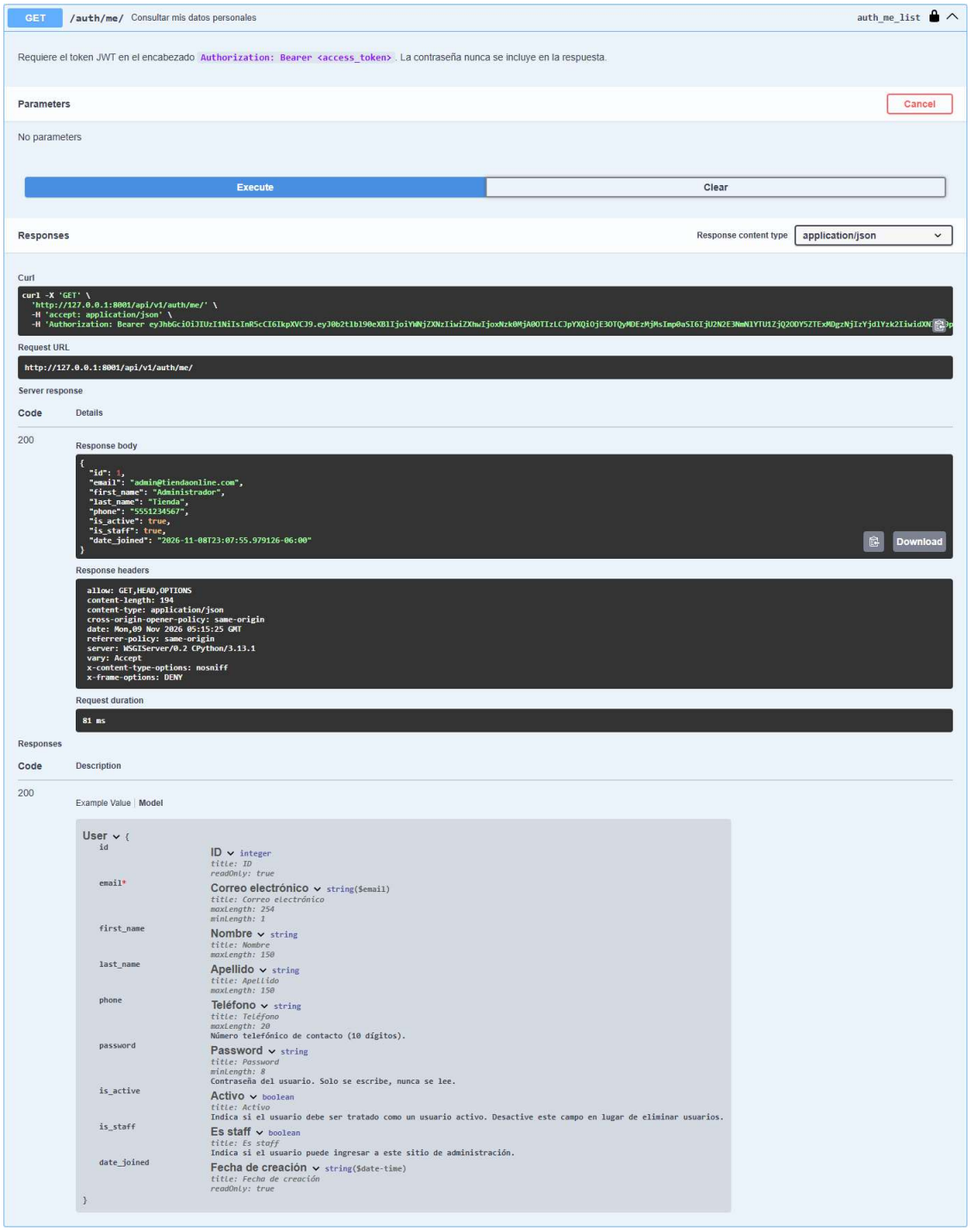
Las capturas se generaron contra http://127.0.0.1:8001 porque el puerto 8000 estaba ocupado por otro proyecto en el equipo. Al ejecutar python manage.py runserver en un equipo libre, la URL es http://127.0.0.1:8000.



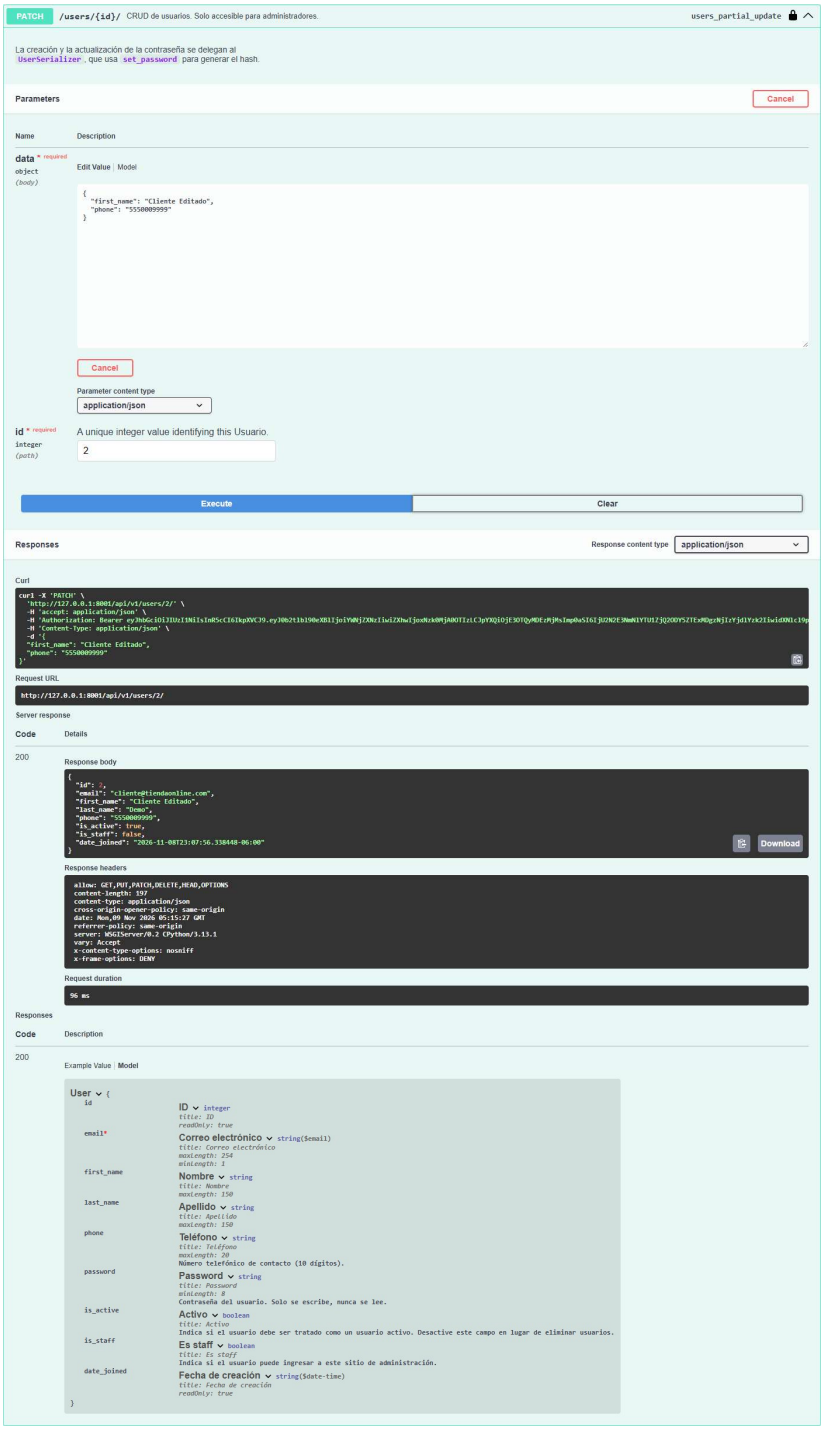
Captura 1. Swagger UI con todos los endpoints documentados. El título es Tienda Online API y se observan las secciones Autenticación, categories, products, Productos y users.



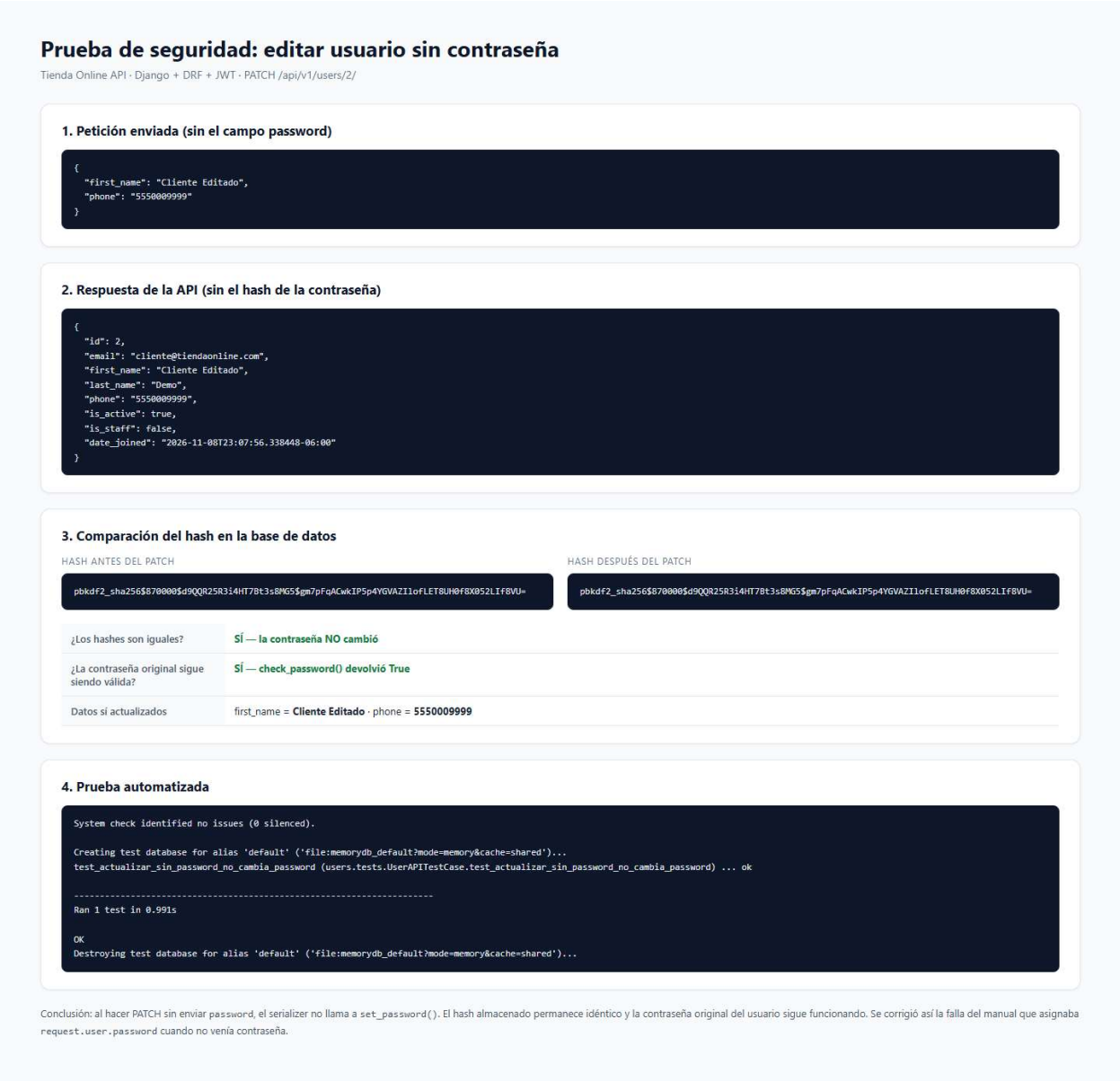
Captura 2. Obtención del token JWT con POST /api/v1/auth/token/ usando email y contraseña. La respuesta incluye refresh y access.



Captura 3. Consulta de datos personales con GET /api/v1/auth/me/ usando el token en Authorize. La respuesta incluye el campo phone y no incluye password.



Captura 4. PATCH /api/v1/users/2/ sin enviar el campo password. La respuesta confirma los datos actualizados y tampoco expone el hash de la contraseña.



Captura 5. Prueba de que editar sin contraseña no la cambia: el hash antes y después del PATCH es idéntico, check_password() devuelve True y la prueba automatizada termina en OK.

6. Ejecución y usuarios de prueba

Comandos para ejecutar el proyecto:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python manage.py migrate
python manage.py seed_demo
python manage.py runserver

# Swagger: http://127.0.0.1:8000/swagger/
# Admin:   http://127.0.0.1:8000/admin/
```

Usuarios de prueba

Rol	Email	Contraseña
Administrador	admin@tiendaonline.com	Admin12345!
Cliente	cliente@tiendaonline.com	Cliente12345!

Estas credenciales son solo para el entorno local de la práctica.

Resultado de las pruebas automatizadas: 17 pruebas OK. Entre ellas se verifica que el listado y /auth/me/ no exponen la contraseña, que crear un usuario guarda el hash, que editar sin contraseña conserva el hash y que editar con contraseña sí la cambia.